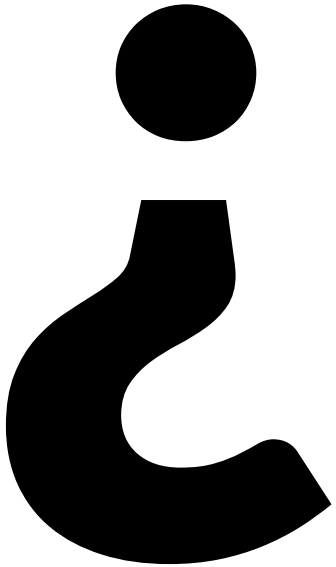
Tipos de datos abstractos

Martín René Vilugrón
mrvilugron@unrn.edu.ar

UNRN
Universidad Nacional
de Río Negro

r33

17 / 2026



Qué es un dato



Del bit al significado

Un conjunto de 0’s y 1’s en memoria no significan nada hasta que una interpretación define sus operaciones válidas

01000001 en la RAM es...

¿El el número entero 65?

¿Es un carácter 'A' en ASCII?

¿Es el inicio de una instrucción de la CPU?

Tomemos un ejemplo cotidiano.

Un control remoto

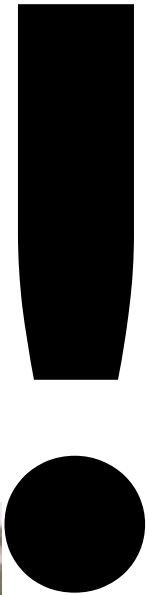
¿Qué pasa cuando apretas Volumen +?

Sube el volumen en la tele…

Pero, **¿Cómo?**



y **no importa**





UNRN | Universidad Nacional de Río Negro

Un microondas

Tiene una perilla de tiempo y un botón de inicio.

El magnetrón bombardea las moléculas de agua con radiación a 2.45 GHz, mientras que la chapa te protege de morir calcinado.



La chapa nos protege

Un cajero automático

Operaciones del lado del público:

```
InsertarTarjeta()
ConsultarSaldo()
ExtraerEfectivo()
```

No podemos abrir la puerta y sacar el dinero…

La abstracción

1. Define qué podemos hacer
2. Oculta complejidad
3. Nos permite ignorar la estructura interna


¿Preguntas?

La caja negra

UNRN Universidad Nacional de **Río Negro**



El usuario conoce **QUÉ** hace la caja.
Solo el autor sabe **CÓMO** lo hace adentro.

Abstracción

Es enfocarse en el "qué" hace un dato, no en el "cómo" está implementado.


¿Preguntas?

Tipos de Datos Abstractos

Una *caja negra* para resolver una parte del dominio del problema.

Que nos abstrae de la forma en la que se resuelve.

Un TDA está compuesto de

1. Un dominio de valores admisibles.
2. Un conjunto de operaciones primitivas que operan sobre dichos valores.
3. El comportamiento de las operaciones está fijado por un contrato, independiente de la representación en memoria.

Definición Canónica (Liskov & Zilles, 1974)


¿ Y en C ?

Logramos la separación con

Interfaz (.h) / Implementación (.c)

Interfaz (.h)

Prototipos de funciones

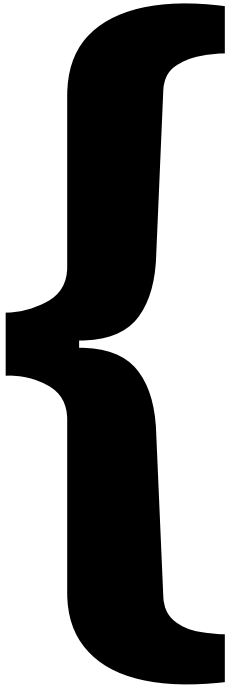
Tipos de datos públicos

Implementación (.c)

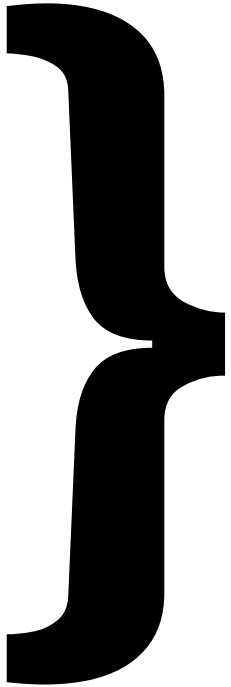
Implementación de los prototipos

Estructuras de datos

Funciones auxiliares

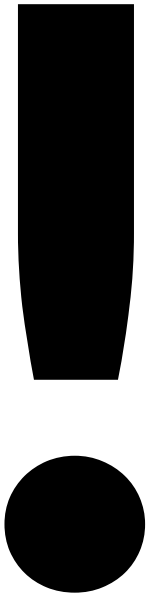


El usuario del TDA





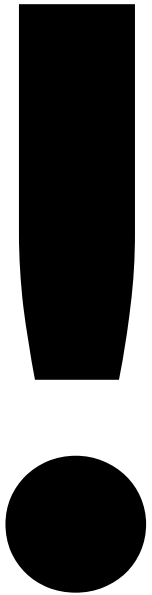
Solo ve el .h

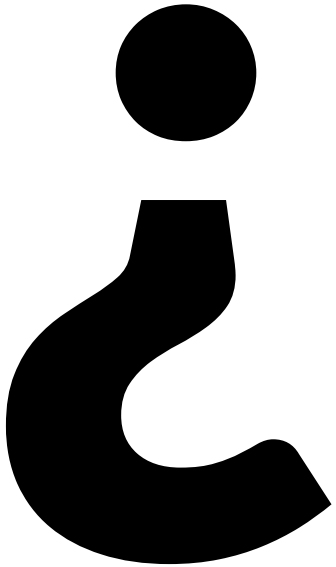


(Si el usuario del TDA solo usa lo que ve en el .h)



La implementación puede cambiar





Cómo se hace esto



Ocultar el `struct`

1 Ocultar la estructura de datos.

Cerrar los Accesos Directos

2

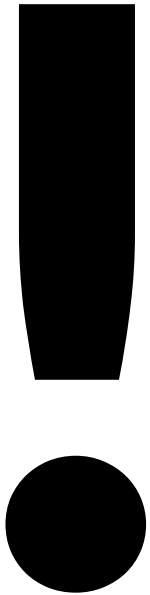
Prohibir acceso directo a campos

Ofrecer Primitivas

3 Operar sólo mediante funciones.



Está faltando algo fundamental



En el .h visible para todos

```
struct arreglo {
    int* datos;
    size_t tam;
};
```

Si el cliente hace:

```
v.tam = 99999999;
```

¡rompe todo!

¿Cómo protegemos la estructura?

Tipos incompletos

```
typedef struct arreglo arreglo_t;
```

El cliente sabe que `arreglo_t` existe, pero desconoce su tamaño y composición.

En el .h, podemos hacer una declaración adelantada (*forward*)

Ocultamos la forma que tiene la estructura

```
sizeof(arreglo_t*) == (generalmente) 8 bytes
sizeof(arreglo_t)  == ERROR DE COMPILACIÓN (Incomplete type)
```

Ni el compilador sabe que hay en un tipo con declaración adelantada!

Y por acá, el detalle solo para el implementador...

```c
#include "arreglo.h"
#include <stdlib.h>

struct arreglo {
    int* datos;
    size_t cantidad;
    size_t capacidad;
};

// más las implementaciones de las funciones del .h
```

En el archivo `arreglo.c`


¿Preguntas?

Para diseñar una interfaz completa y ordenada, clasificamos las primitivas según su **responsabilidad**.

Taxonomía

de operaciones

Constructor - prepara el terreno

Asignación de memoria e inicialización

```
arreglo_t* arreglo_crear(size_t capacidad);
```

Selector - Recupera información

Nos algo de lo que está guardado en la estructura

```
int arreglo_obtener(const arreglo_t* arr, size_t
indice, int* destino);
```

Consultor - Recupera meta-información

Nos da alguna propiedad intrínseca de la estructura; algunas pueden recibir un «predicado»

```
size_t arreglo_largo(const arreglo_t* arr);
bool arreglo_esta_vacio(const arreglo_t* arr);
```

Mutador - Modifica la información

Altera el estado interno de la estructura manteniendo su validez.

```
bool arreglo_insertar(arreglo_t* arr, size_t pos, int valor);
```

Conversor - Crea una estructura similar

A partir del contenido de la estructura actual.

```
char* arreglo_a_cadena(const arreglo_t* arr);
```

Destructor - Libera los recursos

Evitando fugas.

```
void arreglo_destruir(arreglo_t* arr);
```

Esto no es...

¿Orientación a objetos?

UNRN Universidad Nacional de Río Negro

Sí...

(casi)

Caso testigo

```
#ifndef PUNTO_H
#define PUNTO_H

typedef struct punto punto_t; /* Tipo opaco */

punto_t* punto_crear(double x, double y);
double   punto_obtener_x(const punto_t* p);
double   punto_obtener_y(const punto_t* p);
double   punto_distancia_origen(const punto_t* p);
void     punto_destruir(punto_t* p);

#endif
```

Podemos usar la implementación Cartesiana

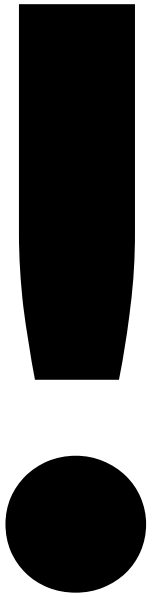
```
struct punto
{
    double x;
    double y;
};
```

O la implementación Polar!

```
struct punto
{
    double radio;
    double angulo;
};
```



Para el usuario de ‘punto..h’ *nada* cambia



{

Lo podemos garantizar con tests

hasta cierto punto

}


¿Preguntas?

Podemos sumar una verificación

invariantes

Invariante

Es una condición lógica sobre el estado interno que siempre debe ser verdadera para que la estructura sea válida.

{ Por ejemplo }

Podemos sumar una función para verificarlas

```
static bool arreglo_invariante(const arreglo_t* v) {
    if (v == NULL) { return false; }
    if (v->datos == NULL) { return false; }
    if (v->capacidad == 0) { return false; }
    if (v->cantidad > v->capacidad) { return false; }
    return true;
}
```

Con todo lo que podemos saber, generalmente de nuestro TDA

¿Cómo usarla?

En la construcción

```
arreglo_t* arreglo_crear(size_t capacidad)
{
    arreglo_t *nuevo;
//construimos
    assert(arreglo_invariante(nuevo));
    return nuevo;
}
```

Nos aseguramos que nuestro código cumpla

En un Selector

```
int arreglo_obtener(const arreglo_t* arr,
                    size_t indice, int* destino)
{
    assert(arreglo_invariante(arr));
// implementación de obtener
}
```

Verificamos como precondición que esté todo bien

En un Mutador

```
bool arreglo_insertar(arreglo_t* arr, size_t pos, int valor)
{
    assert(arreglo_invariante(arr)); // como precondición
//implementación de la inserción.
    assert(arreglo_invariante(arr)); // como poscondición
    return es_correcto;
}
```

Garantizamos que si esta bien, va a salir bien

{ Pueden usar `assert` o el retorno de la función* }


¿Preguntas?

Como se documenta

Constructor - prepara el terreno

```
/**
 * @brief Crea un nuevo arreglo dinámico vacío.
 * @param[in] capacidad Cantidad máxima inicial de elementos.
 * @return Puntero al arreglo creado, o NULL si falla la memoria.
 * @pre capacidad > 0.
 * @post El arreglo queda inicializado con cantidad 0.
 * @note El llamador es responsable de liberarlo con arreglo_destruir().
 */
arreglo_t* arreglo_crear(size_t capacidad);
```

Selector - Recupera información

```
/**
 * @brief Recupera el elemento en la posición indicada.
 * @param[in]  v       Puntero al arreglo.
 * @param[in]  indice  Posición a consultar (base 0).
 * @param[out] destino Dirección donde se copiará el valor leído.
 * @return true si el índice es válido, false en caso contrario.
 * @pre v != NULL y destino != NULL.
 * @pre 0 <= indice < arreglo_largo(v).
 */
bool arreglo_obtener(const arreglo_t* arr, size_t indice, int* destino);
```



Hablar en términos de la abstracción



Revelar la estructura interna y la implementación


¿Preguntas?

TP6

Tipos de datos abstractos

Hasta la próxima

UNRN Universidad Nacional de **Río Negro**